AOIA-Core Import Layout Repair Audit Series v1.0

Sonnet - Gemini/Saphire - Grok - DeepSeek - Kimi
Prepared for AOIA-Core reviewer-readiness archive

Date: 05 June 2026. Branch context: dev/gt-runtime-8-bash-safety-planning. Current checkpoint after Step 1: b4fb53d docs: add reviewer packaging infrastructure.

Purpose: preserve the model-audit record for the import-layout repair decision. This is the second methodology/audit artifact in the AOIA MAHT series. It documents the first transition from a temporary PYTHONPATH-based CI compatibility step toward a reviewer-friendly editable-install workflow.

1. Executive Decision Summary

The audit series converged on a practical conclusion: the temporary PYTHONPATH=runtime:. workflow was valid for Step 1, but should not remain permanent before deeper GT-RUNTIME-9 production work.

The recommended next action is a packaging-only editable-install experiment. It must not touch runtime source files, tests, scripts, executor.py, shell_tools.py, providers, Cloudflare, or browser automation. If successful, the target reviewer flow becomes: python3 -m pip install -e . followed by python3 -m unittest discover -s tests -p test*.py -v.

The key technical lever is setuptools metadata in pyproject.toml: package-dir = {"" = "runtime"}, py-modules = ["main"], and packages.find where = ["runtime"]. DeepSeek and Kimi recommend avoiding an overly narrow include whitelist and inspecting runtime/ contents before committing.

2. Audit Consensus Matrix

Model	Main Position	Actionable Takeaway
Sonnet	PYTHONPATH acceptable only temporarily; target cold-clone / editable-install workflow.	Do not mix import repair with runtime feature work.
Gemini/Saphire	Step 1 is good reviewer packaging, but PYTHONPATH is reviewer friction.	Normalize reviewer workflow before deeper review.
Grok	Reject permanent PYTHONPATH; use packaging-only metadata fix.	Do not refactor tests/runtime in this phase.
DeepSeek	Approve packaging-only repair with amendment: avoid narrow package whitelist.	Use permissive packages.find and verify runtime contents.
Kimi	Approve editable-install mechanism; tested mock layouts.	Document side effect: runtime/ exposed during editable install.

3. Model 1 - Sonnet: CI Import Path Analysis

This section preserves the earlier Sonnet position on the temporary CI decision.

```
AOIA-Core CI Import Path - Option Analysis

The Core Diagnostic

The tests were always run as PYTHONPATH=runtime:. python3 -m unittest discover, as documented in the
README and every reviewer quickstart. The CI file you added runs discovery from root without that env
var. This is a CI configuration gap, not a code defect. That framing matters for reviewer trust: the
fix should match the framing.

Option Comparison

A) PYTHONPATH=runtime in GitHub Actions CI

This exactly replicates the documented validation command. No code changes, no behavior changes, no new
dependencies. The README already shows this invocation, so a reviewer who runs it locally and sees CI
run the same command will have consistent experiences.

Reviewer trust: HIGH - CI matches documented commands.
Runtime behavior change risk: ZERO.
Maintainability: GOOD - one line in the workflow.
CI reliability: HIGH - already known to produce 470 passed, 4 skipped.
Appropriate before NLnet review: YES - this is the correct option for this phase.

B) pyproject.toml path handling (editable install or packages config)

Would require either marking runtime/ as a package root (e.g., pythonpath = ["runtime"] in pytest config,
or [tool.setuptools.packages]), or making the repo pip-installable. For unittest discovery (not
pytest), this doesn't help directly - pyproject.toml doesn't set PYTHONPATH for python -m unittest.
You'd need to either switch to pytest or add a conftest.py. Either introduces new dependencies or
test-runner changes that are outside the reviewer-packaging scope and could introduce subtle behavior
differences.

Reviewer trust: MEDIUM - looks more professional but changes the test invocation from what's documented.
Runtime behavior change risk: LOW but nonzero - switching test runners mid-review is a change.
Maintainability: BETTER long-term but requires a separate audit pass.
Appropriate before NLnet review: NO - scope creep for this phase.

C) Root-level compatibility shims

Adding stub __init__.py or thin wrapper modules at repo root would let root-discovery work. But this
creates two import paths for the same code, which is exactly the kind of ambiguity that causes
reviewer confusion and future bugs. The AOIA cleanup docs already flagged dual-path contamination as
a historical problem.

Reviewer trust: LOW - shims look like something was patched around rather than understood.
Runtime behavior change risk: MEDIUM - dual import paths can cause subtle isinstance failures if the same
class is imported via two different module paths.
Appropriate before NLnet review: NO.

D) Refactor tests to use runtime.* imports

Clean long-term solution: change from tools.executor import ... to from runtime.tools.executor import ...
throughout tests. But this touches many test files, is a substantial change, and requires its own
audit pass. Doing it now under time pressure before NLnet review is the highest-risk option.

Reviewer trust: HIGH if done correctly, but a rushed broad change before review is itself a risk signal.
Runtime behavior change risk: LOW for tests, but touching many files means many opportunities for
accidental edits.
Appropriate before NLnet review: NO - correct eventual fix, separate audited phase.

E) Defer CI entirely

Reviewers will see a repository with CI that either doesn't run or fails. A failing CI badge is worse
than no CI badge. A missing CI file is a gap, not a failure. If the choice is between failing CI and
no CI, remove ci.yml and document validation as manual-only.

Recommendation: Option A

Fix the CI workflow to match the documented validation command exactly. This is a one-line change, zero
behavior risk, and produces the result reviewers expect from reading the README.
```

```
What to record

Add a one-line note to STATUS.md or the cleanliness checkpoint:
CI uses PYTHONPATH=runtime:. to match the documented validation command. Import layout reorganization is
deferred to a separate audited phase after GT-RUNTIME-8.
```

4. Model 1b - Sonnet: Import Layout Repair Structural Warning

This section preserves the later Sonnet-style structural warning that pyproject.toml should eventually become operational rather than decorative.

```
AOIA-Core Import Layout Repair Method Audit - Sonnet Static Analysis

The CI invocation PYTHONPATH=runtime:. python3 -m unittest discover encodes three architectural
decisions:
- source lives in runtime/, not in an installed package;
- root-level modules or tests need the repo root;
- test discovery depends on environment-level sys.path mutation.

Finding F-01 - Dual-root ambiguity.
With both runtime/ and . on PYTHONPATH, module shadowing becomes possible if the same name exists in both
locations.

Finding F-02 - No installed package.
The presence of pyproject.toml signals intent to be installable, but CI never exercises pip install -e .
unless added later.

Finding F-03 - Test discovery pattern.
The pattern test*.py is acceptable for unittest, but actual filenames should be audited.

Finding F-04 - __init__.py/package-mode risk.
unittest discovery can require test modules to be importable depending on start directory and top-level
directory behavior.

Finding F-05 - pyproject package-dir likely absent or incomplete.
If source lives in runtime/ but setuptools uses default discovery, the package discovery may not match
the actual repo layout.

Summary:
The current PYTHONPATH command is a workaround for an unresolved packaging gap, not a stable final
strategy. For NLnet review, the target should be a reproducible cold-clone flow such as: git clone ->
pip install -e . -> python -m unittest discover. This should be treated as a separate import-
layout/packaging repair phase, not mixed with runtime feature work.
```

5. Model 2 - Gemini/Saphire: Reviewer Readiness Audit

```
AOIA-Core Reviewer Readiness Audit
Target Audience: NLnet Technical Evaluation Committee / Open-Source Contributors
Phase Reviewed: STEP 1 (Reviewer-Packaging)
Target Branch: dev/gt-runtime-8-bash-safety-planning

1. Initial Impressions & Strengths

From an external reviewer's perspective, the completion of STEP 1 demonstrates a mature understanding of
open-source hygiene and evaluator requirements.

Documentation Anchors: The presence of CONTRIBUTING.md and STATUS.md immediately signals that AOIA is not
just a personal script repository, but a structured framework ready for external collaboration.
STATUS.md is particularly highly regarded by grant reviewers as it provides an honest map of current
capabilities versus technical debt.

Modern Build Standards: Implementing pyproject.toml is the correct, forward-looking choice for Python
packaging, overriding legacy setup.py approaches.

Automated Verification: The inclusion of .github/workflows/ci.yml assures the reviewer that the local
tests you claim are passing will be continuously verified in a sterile environment.

2. Technical Vulnerability: The Import Layout

The primary point of friction for an open-source evaluator lies in the test execution command:
PYTHONPATH=runtime:. python3 -m unittest discover -s tests -p "test*.py" -v

The Reviewer's Perspective

While 470 passing tests is an excellent metric of stability, relying on an explicit, manual PYTHONPATH
override is a red flag for project encapsulation. It indicates that the module import paths are
deeply coupled to the current working directory and are not organically discoverable by the Python
interpreter.

Friction: An external reviewer pulling the repository expects to run standard commands. Forcing a path
override increases cognitive load and creates fragile onboarding.

Installation Implication: If the tests require runtime:. in the path, it raises questions about how the
package behaves when installed system-wide or in a fresh virtual environment where the repository
structure no longer exists.

3. Recommendations for STEP 2

1. Leverage pyproject.toml for Editable Installs: Ensure pyproject.toml is configured to build the
runtime directory correctly. Running pip install -e . should map the directories so that the test
suite can run natively without environment-variable injection.

2. Test Environment Standardization: Consider a simple Makefile later. A reviewer should ideally type one
command such as make test, but this is not required for this narrow phase.

3. Sanity Check CI: Verify that CI is not artificially injecting PYTHONPATH once the editable-install
flow works. CI should test the package as a user would install it.

4. Conclusion

AOIA-Core has successfully laid structural groundwork for an open-source release. The final barrier to a
pristine external review is normalizing the import architecture to allow zero-configuration or
standard test execution.
```

6. Model 3 - Grok: Red-Team Audit

```
RED TEAM AUDIT - AOIA-Core Import Layout Repair

1. Red-team verdict
Option A (permanent PYTHONPATH) is unacceptable technical debt and a blatant reviewer red flag. It will
be read by NLnet as "we couldn't be bothered to do proper packaging." Any path that touches
runtime/*.py or tests/*.py in this phase is also rejected - it violates the explicit constraints and
starts GT-RUNTIME-9 early. The only defensible move is a minimal, source-untouched packaging metadata
fix that makes pip install -e . + plain unittest work exactly as the current top-level imports
expect.

2. Highest-risk repair path
A (keep PYTHONPATH=runtime:. permanently) or C (bulk refactor of tests to runtime.*). Both lock in the
broken contract, guarantee future breakage when layout is normalized, create massive diff surface,
and will be rejected on sight by any competent reviewer.

3. Lowest-risk repair path
B (configure pyproject.toml so pip install -e . exposes the current top-level modules from under runtime/
without touching a single .py file). This is pure packaging work, already in scope for the reviewer-
packaging phase, produces the exact clean flow Sonnet/Gemini demanded, and leaves zero source debt
for GT-RUNTIME-9.

4. Whether editable install can safely support current imports
Yes. With package-dir = {"" = "runtime"}, package discovery for
tools/adaptive_routing/retrieval/orchestrator, and py-modules = ["main"], pip install -e . can make
import main, from tools.provenance import ..., from adaptive_routing..., import retrieval.linux,
import orchestrator... resolve identically to PYTHONPATH=runtime:. today. No test changes, no runtime
changes, no new behavior.

5. Whether converting tests to runtime.* is safer or riskier
Riskier in this phase. It requires editing many import sites, risks incomplete refactors, string-based
imports, lazy imports, and dynamic loading issues. Packaging the existing top-level imports gives the
reviewer experience with zero source edits.

6. Whether runtime internal imports should be touched in the same phase
No. Changing internal imports can alter __package__, import side-effects, relative resolution, and module
caching. That work belongs strictly in a later import-namespace normalization phase.

7. Minimal safe repair sequence
- Edit only pyproject.toml: add package-dir / packages.find / py-modules.
- Edit only CI: replace PYTHONPATH line with pip install -e . and plain unittest.
- Update CONTRIBUTING.md and STATUS.md with the reviewer flow.
- Push only after validation.

8. What experiments Codex must run before editing
Fresh editable install, direct import checks for main/tools/adaptive_routing/retrieval/orchestrator,
plain unittest expecting 470 passed / 4 skipped, compileall, and confirmation of no new dependencies
or source edits.

9. What files must not be touched
Anything under runtime/, tests/, protected executor/shell tools/main, providers, Cloudflare, browser
automation, and scripts.

10. What validation must pass
pip install -e . succeeds; direct imports work without PYTHONPATH; python -m unittest discover reports
470 passed / 4 skipped; compileall passes; CI is green.

11. What would make me reject the repair
Any remaining PYTHONPATH in CI/docs, any runtime/test edit, incomplete pyproject that doesn't make pip
install -e . work, introduction of pytest/lint/coverage/dependencies, or a large import-refactor
commit.

12. Risk score
7/10. Underlying layout debt still exists and will need a later normalization, but this phase can stop
compounding the debt.

13. Final recommendation
Do B only. Reject A, C, D, and E as standalone strategies. A combination is acceptable only if it is
literally B plus documentation; even empty __init__.py files are unnecessary unless proven by the
repo.
```

7. Model 4 - DeepSeek: Corrected Packaging-only Audit

```
DeepSeek - Final Packaging-only Import Layout Repair Audit

Audit Verdict: APPROVE with one critical amendment.

What's correct:
- package-dir mapping {"" = "runtime"} correctly relocates the import root.
- py-modules = ["main"] properly exposes the top-level module.
- namespaces = false prevents subtle namespace-package bugs.
- Static version 0.0.0 avoids setuptools_scm entanglement.
- exclude pattern ["tests*"] prevents package pollution.

Critical amendment required:
The include whitelist is dangerously incomplete. The proposed test imports cover tools.provenance,
adaptive_routing, retrieval.linux and orchestrator, but runtime/ may contain other top-level packages
or utility modules. A narrow include list could silently omit them from the installed package.

Recommended fix:

Option A (safer, more permissive):

[tool.setuptools.packages.find]
where = ["runtime"]
exclude = ["tests*", "__pycache__*"]
namespaces = false

Remove the include entirely and let setuptools discover everything except tests.

Option B (explicit, requires verification):
Keep the include list but add a catch-all or verify exact runtime contents before committing.

What to verify before pushing:
1. List all runtime/ contents to ensure no orphaned packages.
2. After pip install -e ., validate import main, tools.provenance and the other target imports.
3. Ensure pip install -e . does not create unintended namespace behavior.

CI update should be:
- run: pip install -e .
- run: python -m unittest discover -s tests -p "test*.py" -v

Remove PYTHONPATH entirely. If tests pass, the repair is complete.

Summary:
The approach is correct. Fix the over-restrictive package discovery, verify no orphaned modules, and this
becomes a clean reviewer-friendly flow. Proceed with the permissive discovery option unless there is
a specific reason to whitelist.
```

8. Model 5 - Kimi: Packaging Mechanism Audit

```
Kimi - AOIA-Core Final Packaging-only Import Layout Repair Audit

Verdict: APPROVED for execution with documented caveats.

Mechanism Validated

The proposed pyproject.toml configuration works through setuptools' editable install mechanism:

1. package-dir = {"" = "runtime"} maps the root package namespace to runtime/
2. During pip install -e ., setuptools creates a .pth file pointing to runtime/
3. This adds runtime/ to sys.path, making all modules/packages within importable at top-level
4. py-modules = ["main"] explicitly registers main.py as a top-level module
5. packages.find with where = ["runtime"] discovers packages under runtime/

Experimental Results

Mock project structures matching AOIA-Core's layout were tested with three configurations:
- Minimal configuration: install success, unittest pass.
- Proposed configuration with include/exclude: install success, unittest pass.
- Edge case with non-package dirs in runtime/: success with side-effect noted.

All target imports validated successfully:
- import main -> runtime/main.py
- from tools import provenance -> runtime/tools/provenance.py
- from adaptive_routing import config_loader -> runtime/adaptive_routing/config_loader.py
- from retrieval.linux import ... -> runtime/retrieval/linux/__init__.py
- from orchestrator import knowledge_router -> runtime/orchestrator/knowledge_router.py

Critical Findings

1. py-modules = ["main"] is redundant but harmless and explicit.
2. include/exclude lists are partially effective; during editable install, the .pth file exposes runtime/
broadly.
3. Non-package directories in runtime/ become importable as implicit namespace packages. Risk is low if
runtime/ contains only intended packages.

Recommendations

1. Approve the proposed pyproject.toml; it achieves the goal:
pip install -e .
python -m unittest discover -s tests -p "test*.py" -v

2. Consider simplification if all subdirectories of runtime/ are intended packages:

[tool.setuptools]
package-dir = {"" = "runtime"}

[tool.setuptools.packages.find]
where = ["runtime"]
exclude = ["tests*"]
namespaces = false

3. Update CI from PYTHONPATH command to:
pip install -e .
python3 -m unittest discover -s tests -p "test*.py" -v

4. Document in STATUS.md that this is a transitional packaging measure; long-term goal remains proper
package namespacing.

5. Add to CONTRIBUTING.md: Do not add non-package directories to runtime/ because runtime/ is exposed as
package/module root during editable install.

Risk Assessment

If runtime/ contains only intended packages: LOW.
If runtime/ contains non-package directories such as templates/static/etc.: MEDIUM.

Final Verdict

APPROVED. The packaging-only repair is sound, tested, and achieves the target reviewer-friendly flow. No
source-code edits required. Execute the pyproject.toml and CI changes as proposed, with documented
caveats.
```

9. Derived Execution Plan for Codex

The next Codex task should be experimental-first, not commit-first. Codex should test editable install locally and commit only if the experiment proves that plain unittest works without PYTHONPATH.

```
Allowed files only:
- pyproject.toml
- .github/workflows/ci.yml
- STATUS.md
- CONTRIBUTING.md

Forbidden files:
- runtime/*.py
- tests/*.py
- scripts/*.py
- executor.py
- shell_tools.py
- main.py
- providers/
- Cloudflare-related files
- browser automation code

Target pyproject core:
[build-system]
requires = ["setuptools >= 61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "aoia-core"
version = "0.0.0"
description = "Local-first pre-execution inspection and audit framework for AI-assisted commands"
requires-python = ">=3.12"
dependencies = []

[tool.setuptools]
package-dir = {"" = "runtime"}
py-modules = ["main"]

[tool.setuptools.packages.find]
where = ["runtime"]
exclude = ["tests*"]
namespaces = false

Validation target:
python3 -m pip install -e .
python3 -m compileall runtime tests scripts
python3 -m unittest discover -s tests -p "test*.py" -v

Expected result:
Ran 470 tests, OK (skipped=4)
```

10. Open Caveats and Future Work

This repair is transitional. It does not yet rename imports into a clean long-term AOIA namespace such as aoia_core.*. It preserves the current top-level import contract while replacing a manual PYTHONPATH override with an editable-install workflow.

A later audited phase should consider real namespace normalization after NLnet reviewer-readiness is stabilized. That later phase may include runtime.* or aoia_core.* imports, package boundaries, and a cleaner application entrypoint. It must not be mixed with the current packaging-only experiment.

The 4 skipped tests should still be identified and documented in a separate skipped-tests audit. This was flagged by prior methodology review as an unresolved quality signal.

11. Final Recommendation

Proceed to Codex with a packaging-only editable-install experiment. If it succeeds, commit and push only the four allowed files. If it fails, roll back and preserve the failure as evidence for the next MAHT report.

This is consistent with AOIA's emerging MAHT principle: models generate hypotheses; experiments decide.